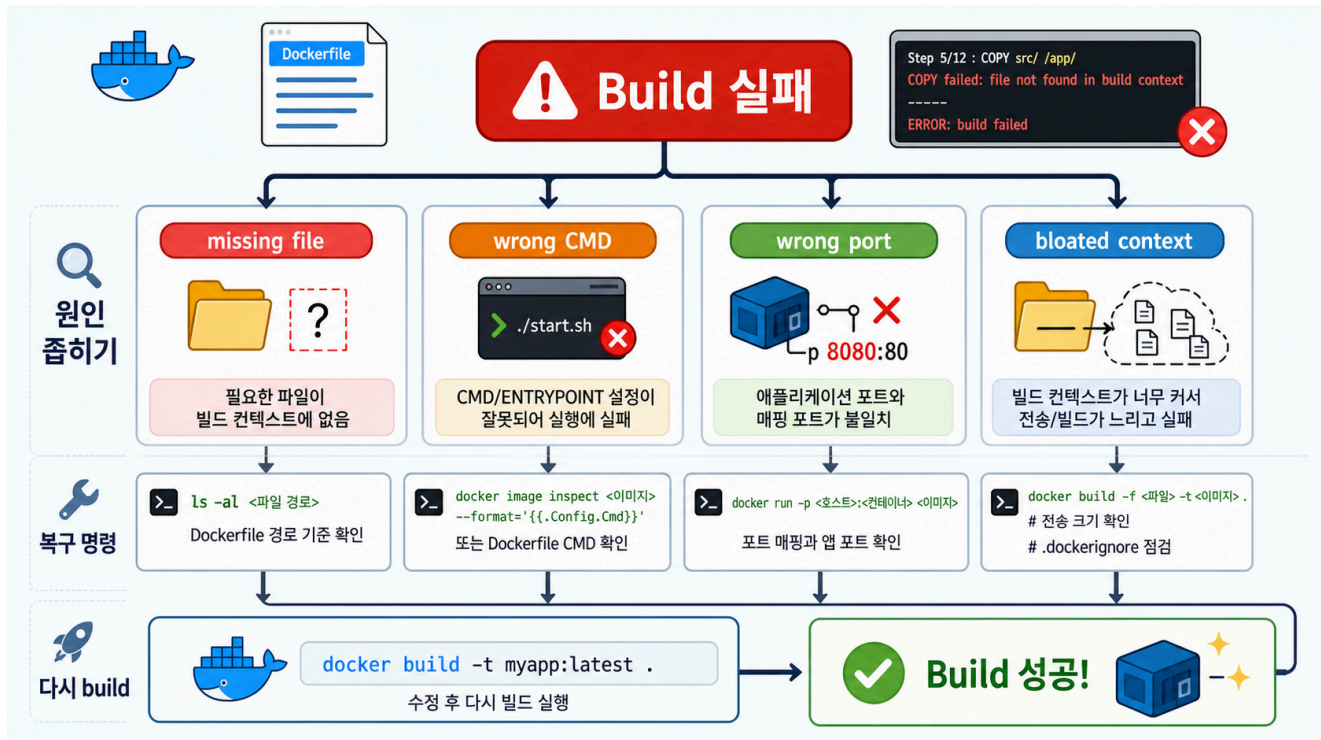


6교시: Failure drill - 실패 출력으로 원인 찾기



수업 목표

- missing file, wrong CMD, wrong port, bloated context를 실제 실패 출력으로 구분한다.
- 실패를 build 단계, run 단계, verify 단계로 나눈다.
- 에러 문구에서 힌트를 찾고, 첫 확인 명령과 복구 명령을 연결한다.

개념 설명

Docker가 안 돼요는 원인이 아니다. 먼저 마지막으로 성공한 단계가 어디인지 나눠야 한다. build가 실패했다면 Dockerfile과 build context를 본다. run에서 container가 죽었다면 image tag, container name, CMD, logs를 본다. HTTP verify만 실패했다면 host port와 container port mapping을 본다.

실패 실습에서는 일부러 망가뜨리는 명령보다 **망가졌을 때 출력되는 문구**가 더 중요하다. 초보자는 에러 메시지를 외우지 않아도 된다. 대신 에러 안에서 COPY, not found, PORTS, Exited, connection refused, Empty reply 같은 힌트를 찾아야 한다.

실패 1: Missing file - build 단계 실패

재현 명령

```
cd /mnt/d/paperclip
cp -r week2/day3/labs/static-site week2/day3/labs/static-site-broken
rm -f week2/day3/labs/static-site-broken/index.html
cd week2/day3/labs/static-site-broken
docker build -t paperclip-static-site:broken . || true
```

예상 실패 출력

Docker/BuildKit 버전에 따라 문구는 조금 다르지만 다음 패턴을 찾는다.

```
=> ERROR [3/4] COPY index.html ./index.html
-----
failed to calculate checksum of ref ...
"/index.html": not found
```

또는:

```
COPY failed: file not found in build context or excluded by .dockerignore:
stat index.html: file does not exist
```

출력에서 찾을 힌트

힌트	의미
ERROR [3/4] COPY index.html	Dockerfile의 COPY index.html 단계에서 실패
not found	source file이 build context 안에 없음
build context	container 실행 전 build 입력 문제

첫 확인 명령

```
pwd
ls -la
sed -n '1,120p' Dockerfile
find . -maxdepth 1 -type f | sort
```

복구 명령

```
cd /mnt/d/paperclip
cp week2/day3/labs/static-site/index.html week2/day3/labs/static-site-broken/index.html
cd week2/day3/labs/static-site-broken
docker build -t paperclip-static-site:broken-fixed .
```

실패 2: Wrong port - verify 단계 실패

재현 명령

```
cd /mnt/d/paperclip
docker run -d --name paperclip-day3-static-wrong -p 18084:8080 paperclip-static-site:day3 || true
curl -I http://localhost:18084 || true
docker ps -a --filter name=paperclip-day3-static-wrong
```

예상 실패 출력

```
curl: (52) Empty reply from server
```

또는 환경에 따라:

```
curl: (7) Failed to connect to localhost port 18084
```

docker ps에서는 다음처럼 보일 수 있다.

```
PORTS
0.0.0.0:18084->8080/tcp
```

출력에서 찾을 힌트

힌트	의미
18084->8080/tcp	host 18084가 container 8080으로 연결됨
Dockerfile EXPOSE 80	nginx는 container 내부 80을 기대
curl 실패	build가 아니라 host에서 접근하는 verify 단계 문제

첫 확인 명령

```
docker ps -a --filter name=paperclip-day3-static-wrong
sed -n '1,120p' week2/day3/labs/static-site/Dockerfile
```

복구 명령

```
docker rm -f paperclip-day3-static-wrong || true
docker run -d --name paperclip-day3-static-fixed -p 18084:80 paperclip-static-site:day3
curl -I http://localhost:18084
```

Expected recovery:

```
HTTP/1.1 200 OK
```

실패 3: Wrong CMD - run 단계 실패

재현 명령

```
cd /mnt/d/paperclip/week2/day3/labs/static-site
awk '{ if ($1 == "CMD") print "CMD [\"nginx-bad-command\"]"; else print $0 }'
Dockerfile > Dockerfile.bad-cmd
docker build -f Dockerfile.bad-cmd -t paperclip-static-site:bad-cmd .
docker run -d --name paperclip-day3-bad-cmd paperclip-static-site:bad-cmd ||
true
docker ps -a --filter name=paperclip-day3-bad-cmd
docker logs paperclip-day3-bad-cmd --tail 30 || true
```

예상 실패 출력

docker ps -a:

```
STATUS
Exited (127) ...
```

docker logs:

```
exec: "nginx-bad-command": executable file not found in $PATH
```

출력에서 찾을 힌트

힌트	의미
Exited	container process가 유지되지 못하고 종료됨
executable file not found	CMD 또는 ENTRYPOINT 명령이 잘못됨
\$PATH	image 안에서 해당 command를 찾지 못함

첫 확인 명령

```
docker ps -a --filter name=paperclip-day3-bad-cmd
docker logs paperclip-day3-bad-cmd --tail 30
docker image inspect paperclip-static-site:bad-cmd --format "{{json
.Config.Cmd}}"
```

복구 명령

```
docker rm -f paperclip-day3-bad-cmd || true
docker image rm paperclip-static-site:bad-cmd || true
rm -f /mnt/d/paperclip/week2/day3/labs/static-site/Dockerfile.bad-cmd
# 원래 Dockerfile의 CMD는 다음이어야 한다.
# CMD ["nginx", "-g", "daemon off;"]
```

실패 4: Bloated context - build는 되지만 느리고 위험한 상태

재현 명령

```
cd /mnt/d/paperclip/week2/day3/labs/static-site
mkdir -p node_modules __pycache__ dist build coverage tmp
printf "DO_NOT_COMMIT_TOKEN=example" > .env
printf "large dependency placeholder" > node_modules/example.txt
printf "compiled output" > dist/app.bundle.js
du -sh .
find . -maxdepth 2 -type f | sort
sed -n '1,160p' .dockerignore
```

예상 출력

```
.env
node_modules/example.txt
```

```
dist/app.bundle.js
__pycache__/...
```

.dockerignore에 다음 패턴이 있어야 한다.

```
.env
.env.*
__pycache__/
node_modules/
dist/
build/
coverage/
tmp/
```

출력에서 찾을 힌트

힌트	의미
.env	secret이 context에 섞일 위험
node_modules	context 크기와 platform 의존성 위험
dist, build	의도하지 않은 build output 포함 위험
du -sh . 증가	build context 전송과 scan 시간이 늘어날 수 있음

복구 명령

```
rm -rf .env node_modules __pycache__ dist build coverage tmp
sed -n '1,160p' .dockerignore
```

RCA 표

실패 유형	단계	대표 출력	첫 확인 명령	수정 방향
missing file	build	COPY ... not found	ls, Dockerfile, find	source path/context 복구
wrong CMD	run	Exited, executable file not found	docker logs, inspect .Config.Cmd	start command 수정
wrong port	verify	curl failed, 18084->8080/tcp	docker ps, Dockerfile EXPOSE	container port 기준 publish
bloated context	build hygiene	.env, node_modules, 큰 du -sh	.dockerignore, find, du	제외 규칙 추가/정리

핵심 포인트

실패 분석의 첫 질문은 어느 단계까지 성공했는가다. 하지만 그 다음 질문은 출력에서 어떤 힌트를 찾았는가다. 에러 메시지를 읽으면 build/run/verify 중 어디를 봐야 할지 좁힐 수 있다.

다음 연결

다음 교시는 정상 image에 tag를 붙이고 registry에 올려도 되는지 gate를 확인한다.